МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

Кафедра «Интеллектуальные информационные технологии»

ЛАБОРАТОРНАЯ РАБОТА №4

по дисциплине «ОСНОВЫ МАШИННОГО ОБУЧЕНИЯ» за 5 семестр

Тема: «Введение в нейронные сети:

построение многослойного перцептрона»

Выполнил:

студент 3 курса

группы АС-67

Иванович Д. И.

Проверил:

Крощенко А. А.

2026

Цель работы: построить, обучить и оценить **многослойный перцептрон (MLP)** для решения задачи классификации

Общее задание:

1. Импорт библиотек и подготовка данных

• импортируйте torch, torch.nn, torch.optim, а также sklearn для загрузки данных и их предобработки;

• загрузите датасет, выполните стандартизацию (StandardScaler) и кодирование признаков;

• разделите данные на обучающую и тестовую выборки;

• преобразуйте данные (признаки и метки) в тензоры PyTorch: torch.tensor(X_train, dtype=torch.float32).

2. Определение архитектуры нейронной сети

• создайте класс, наследуемый от torch.nn.Module;

• в методе __init__ определите все слои, которые будете использовать (например, nn.Linear, nn.ReLU, nn.Dropout);

• в методе forward опишите последовательность применения слоев к входным данным.

3. Инициализация модели, функции потерь и оптимизатора

• создайте экземпляр вашей модели: model = MLP();

• определите функцию потерь. Для бинарной классификации

• используйте nn.BCEWithLogitsLoss, для многоклассовой –

• nn.CrossEntropyLoss;

• определите функцию потерь. Для бинарной классификации используйте nn.BCEWithLogitsLoss, для многоклассовой – nn.CrossEntropyLoss;

4. Написание цикла обучения (Training Loop)

• запустите цикл на определенное количество эпох;

• внутри цикла:

1) переведите модель в режим обучения: model.train();

2) сделайте предсказание (forward pass): y_pred = model(X_train);

3) рассчитайте потери (loss): loss = criterion(y_pred, y_train);

4) обнулите градиенты: optimizer.zero_grad();

5) выполните обратное распространение ошибки: loss.backward();

6) сделайте шаг оптимизации: optimizer.step().

5. Оценка модели (Evaluation)

• переведите модель в режим оценки: model.eval();

• используйте with torch.no_grad():, чтобы отключить расчет градиентов;

• сделайте предсказания на тестовых данных;

• преобразуйте выходные данные (логиты) в предсказанные классы (например, с помощью torch.argmax или проверки порога > 0);

• рассчитайте метрики (accuracy, f1-score и т.д.), используя sklearn.metrics.

Вариант 3 Классификация качества вина

• Набор данных: Wine Quality

• Задача: классифицировать вино на "хорошее" (оценка >= 7) и "обычное" (бинарная классификация).

• Архитектура:

- входной слой;
- два скрытых слоя по 12 нейронов в каждом (ReLU);
- выходной слой с 1 нейроном (Sigmoid).

• Эксперимент: увеличьте количество эпох обучения в два раза и посмотрите, привело ли это к улучшению F1-score.

Код программы:

```
import torch
from torch import nn
from torch import optim


import pandas as pd
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score, f1_score

device = "cuda" if torch.cuda.is_available() else "cpu"
print("Платформа обучения: ", device)

class MLP(nn.Module):
    def __init__(self, in_features: int) -> None:
        super().__init__()
        # Архитектура по заданию: два скрытых слоя по 12 нейронов (ReLU), выход 1 нейрон
        self.linear_stack = nn.Sequential(
            nn.Linear(in_features=in_features, out_features=12),
            nn.ReLU(),
            nn.Linear(in_features=12, out_features=12),
            nn.ReLU(),
            nn.Linear(in_features=12, out_features=1),
        )

    def forward(self, x):
        return self.linear_stack(x)

def train_and_evaluate(epochs: int, data) -> float:
    X_train_t, X_test_t, y_train_t, y_test_t = data

    torch.manual_seed(42)
    model = MLP(in_features=X_train_t.shape[1]).to(device)

    # Считаем количество нормальных и хороших вин в тренировочной выборке
    num_neg = (y_train_t == 0).sum().item()  # обычные
    num_pos = (y_train_t == 1).sum().item()  # хорошие
    # Вес для положительного класса = (кол-во отрицательных) / (кол-во положительных)
    pos_weight = torch.tensor([num_neg / num_pos], dtype=torch.float32).to(device)
```

```
    # Передаем вес в функцию потерь
    criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)
    optimizer = optim.Adam(model.parameters(), lr=0.001)

    print("=" * 55)
    print(f"\nОбучение модели. Количество эпох: {epochs}")
    print(f"Дисбаланс классов: отрицательных {num_neg}, положительных
{num_pos}")
    print(f"Применен pos_weight = {pos_weight.item():.2f}")
    print("\tФункция потерь: ", criterion)
    print("=" * 55)

    for epoch in range(epochs):
        model.train()
        y_pred = model(X_train_t)
        loss = criterion(y_pred, y_train_t)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        if (epoch + 1) % 50 == 0 or epoch == 0:
            print(f"Эпоха {epoch + 1}/{epochs}, loss = {loss.item():.4f}")

    print("\nОценка модели нейронной сети: ")
    model.eval()
    with torch.no_grad():
        logits = model(X_test_t)
        predicted_classes = (logits > 0).float()

    y_true = y_test_t.cpu().numpy().ravel()
    y_hat = predicted_classes.cpu().numpy().ravel()

    # zero_division=0 убирает предупреждения, если модель всё же не
предскажет класс
    print(classification_report(y_true, y_hat, target_names=['обычное (<7)',
'хорошее (>=7)'], zero_division=0))
```

```
    f1 = f1_score(y_true, y_hat)
    print(f"F1-score: {f1:.4f}")
    return f1

def solution() -> None:
    try:
        df = pd.read_csv('winequality-white.csv', sep=';')
    except FileNotFoundError:
        print("Ошибка: Файл 'winequality-white.csv' не найден.")
        return

    y = (df['quality'] >= 7).astype(int)
    X = df.drop(columns=['quality'])

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, train_size=0.7, random_state=42, stratify=y)

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    X_train_t = torch.tensor(X_train_scaled, dtype=torch.float32).to(device)
    X_test_t = torch.tensor(X_test_scaled, dtype=torch.float32).to(device)
    y_train_t = torch.tensor(y_train.values, dtype=torch.float32).view(-1,
1).to(device)
    y_test_t = torch.tensor(y_test.values, dtype=torch.float32).view(-1,
1).to(device)

    data = (X_train_t, X_test_t, y_train_t, y_test_t)

    base_epochs = 100
    double_epochs = base_epochs * 2

    print("\n--- БАЗОВЫЙ ЭКСПЕРИМЕНТ ---")
    f1_base = train_and_evaluate(base_epochs, data)

    print("\n\n--- ЭКСПЕРИМЕНТ С УВЕЛИЧЕНИЕМ ЭПОХ В 2 РАЗА ---")
```

```
    f1_double = train_and_evaluate(double_epochs, data)


    print("\n" + "=" * 55)
    print("Сравнение F1-score:")
    print(f"\t{base_epochs} эпох : {f1_base:.4f}")
    print(f"\t{double_epochs} эпох : {f1_double:.4f}")


    if f1_double > f1_base:
        print("\nВывод: Увеличение количества эпох ПРИВЕЛО к улучшению F1-
score.")
    elif f1_double < f1_base:
        print("\nВывод: Увеличение количества эпох НЕ привело к улучшению F1-
score (возможно переобучение).")
    else:
        print("\nВывод: Увеличение количества эпох не изменило F1-score.")
    print("=" * 55)


if __name__ == "__main__":
    solution()
```

```
Платформа обучения:  cpu

--- БАЗОВЫЙ ЭКСПЕРИМЕНТ ---
============================================================

Обучение модели. Количество эпох: 100
Дисбаланс классов: отрицательных 2686, положительных 742
Применен pos_weight = 3.62
        Функция потерь:  BCEWithLogitsLoss()
============================================================
Эпоха 1/100, loss = 1.0962
Эпоха 50/100, loss = 1.0478
Эпоха 100/100, loss = 0.9636

Оценка модели нейронной сети:
               precision    recall  f1-score   support

 обычное (<7)       0.90      0.65      0.75      1152
хорошее (>=7)       0.37      0.75      0.49       318

     accuracy                           0.67      1470
    macro avg       0.64      0.70      0.62      1470
 weighted avg       0.79      0.67      0.70      1470

F1-score: 0.4932
```

Рис. 1. Результат программы с обучением нейронной сети с 100 эпох

```
--- ЭКСПЕРИМЕНТ С УВЕЛИЧЕНИЕМ ЭПОХ В 2 РАЗА ---
============================================================

Обучение модели. Количество эпох: 200
Дисбаланс классов: отрицательных 2686, положительных 742
Применен pos_weight = 3.62
        Функция потерь:  BCEWithLogitsLoss()
============================================================
Эпоха 1/200, loss = 1.0962
Эпоха 50/200, loss = 1.0478
Эпоха 100/200, loss = 0.9636
Эпоха 150/200, loss = 0.8598
Эпоха 200/200, loss = 0.7982

Оценка модели нейронной сети:
               precision    recall  f1-score   support

 обычное (<7)       0.91      0.77      0.84      1152
хорошее (>=7)       0.47      0.72      0.57       318

     accuracy                           0.76      1470
    macro avg       0.69      0.75      0.70      1470
 weighted avg       0.81      0.76      0.78      1470

F1-score: 0.5679

============================================================
Сравнение F1-score:
        100 эпох : 0.4932
        200 эпох : 0.5679

Вывод: Увеличение количества эпох ПРИВЕЛО к улучшению F1-score.
============================================================
```

Рис. 2. Результат программы с обучением нейронной сети с 200 эпох

Вывод: в ходе выполнения работы был обучен многослойный перцептрон для бинарной классификации качества вина. Исходный датасет обладает сильным дисбалансом классов (хороших вин около 22%). Без специальных мер модель предсказывала только мажоритарный класс, давая F1-score = 0.